Reading Input in C++ (Addenda)

Computer Programming 1

Reading lines from input streams



- Reading line by line with static allocation
- Reading char by char
- Reading line by line with dynamic allocation

Reading with `getline(char*, size)`




```
#include <iostream>

int main() {
    char line[20];
    std::cin.getline(line, sizeof(line));
    std::cout << line << '\n';
}
```

- The array has room for 20 characters, including the terminating `'\0'`.
- The maximum line length is 19 characters.
- The newline is consumed but not stored.

What happens when the line is too long?




```
char line[10];
std::cin.getline(line, sizeof(line));
```

- Only part of the line fits in the array
- `getline` returns something equivalent to zero/false
- The stream sets its `failbit`.
- Further input operations fail until the stream state is cleared.
- The rest of the line may still be waiting in the input stream.

Simple solution



A naive solution is just stopping the program. This will not work in many cases. (Suppose the program asks the user the coordinates for landing and the user enters too many characters :-))

```
#include <iostream>

int main() {
    char line[20];
    while (std::cin.getline(line, sizeof(line))) {
      std::cout << line << '\n';
    }
}
```

Recovering from an overlong line



- Better: detect the error, clear the stream state, and discard the rest of the line.
- `clear()` resets the stream error state.
- `ignore()` discards characters up to and including the next newline.
- The application can continue reading subsequent lines.

```
#include <iostream>
#include <limits>

int main() {
  char line[10];

  while (true) {
    std::cin.getline(line, sizeof(line));

    if (std::cin.fail()) {
      std::cin.clear();
      std::cin.ignore(
         std::numeric_limits<std::streamsize>::max(),
         '\n'
      );
      std::cout << "(Too long) read: " << line << std::endl;
    } else if (std::cin.eof()) {
      break;
    } else {
      std::cout << line << '\n';
    }
  }
}
```

Reading one character at a time with `getc()`



The C function `getc()` reads a single character from a stream.

```
#include <cstdio>

int main() {
    int c;

    while ((c = getc(stdin)) != EOF) {
        putchar(c);
    }
}
```

- `stdin` is the standard input stream.
- `getc()` returns the next character, or `EOF` at end-of-file.
- The return value must be stored in an `int`, not a `char`, to distinguish all character values from `EOF`.

Reading a line with `getc()`



- If the manipulates char by char (e.g., uppercase), this is the perfect solution: we can read characters until we encounter a newline
- We control exactly when to stop reading
- We can process each character as it arrives

```
#include <cstdio>

int main() {
    int c;

    while ((c = getc(stdin)) != EOF && c != '\n') {
        putchar(c);
    }

    putchar('\n');
}
```

Using `getc` to build an array



- We can use `getc` to control what to do when a line is too long
- The code reports a line with nine characters as ”too long”, eve though it could fit

```
#include <iostream>
using namespace std;
const int SIZE = 10;

int main() {
  char line[SIZE]; char c; int index = 0;

  while (cin.get(c)) {
    if (index == SIZE - 1) {
      line[index] = '\0';
      cout << "Read up to: " << line << endl;
      index = 0; // we could discard up to the newline
    }
    else if (c == '\n') {
      line[index] = '\0';
      cout << "Line is ok: " << line << endl;
      index = 0;
    }

    line[index++] = c;
  }
}
```

Introducing `std::string`



- `std::string` can grow as needed.
- No fixed array size is needed.
- The string grows as characters are read.
- The newline is consumed but not stored.
- The line can be much longer than a small fixed buffer.

```
#include <iostream>
#include <string>

int main() {
    std::string line;

    std::getline(std::cin, line);

    std::cout << line << '\n';
}
```

Reading multiple lines with `std::string`



- This is the usual way to read lines from standard input
- The loop continues until input ends or an error occurs
- Each successful read produces one complete line
- No manual recovery is needed just because a line exceeds an initial capacity

```
#include <iostream>
#include <string>

int main() {
    std::string line;

    while (std::getline(std::cin, line)) {
        std::cout << "Length: "
                  << line.size() << '\n';
        std::cout << line << '\n';
    }
}
```